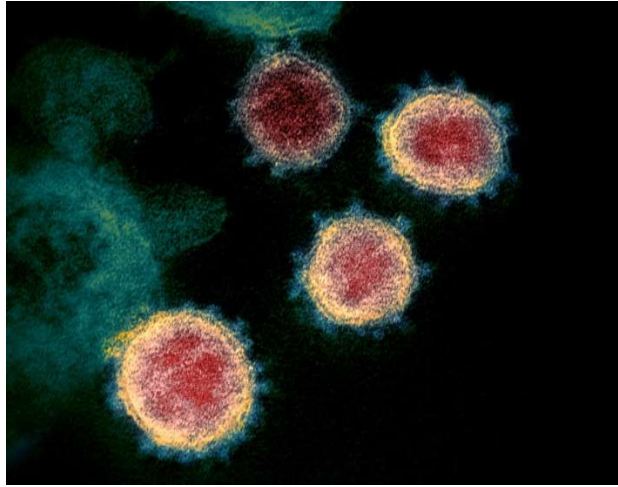


Assembly and Annotation of the SARS-CoV-2 Genome

Introduction



This is the first assignment in which we will use data and software produced by others to draw conclusions about the SARS-CoV-2 virus that has caused the COVID-19 pandemic. In this sense, we are “research parasites”, a term coined by a now infamous [2016 editorial](#) in the *New England Journal of Medicine*:

A ... concern held by some is that a new class of research person will emerge — people who had nothing to do with the design and execution of the study but use another group’s data for their own ends, possibly stealing from the research productivity planned by the data gatherers, or even use the data to try to disprove what the original investigators had posited. There is concern among some front- line researchers that the system will be taken over by what some researchers have characterized as “research parasites.”

We start our study of SARS-CoV-2 by assembling its genome. This task was first completed on [January 25, 2020](#) using a sample collected from the first known human patient.

Many patient DNA or RNA samples taken from swabs are **metagenomic**, meaning that they capture genetic material not only from the virus but also other viruses, bacteria, and even human cells. This means that researchers either have to apply an algorithm that is able to assemble genomes from information derived from multiple species, or they need to perform additional laboratory work to isolate the viral information. The researchers who assembled the first SARS-CoV-2 genome did the former, wrangling a 30,000 base pair genome out of a file consisting of [8 billion base pairs](#), most of which do not derive from SARS-CoV-2. If genome assembly is like assembling a puzzle, metagenome assembly is like assembling multiple similar puzzles from a jumbled set of pieces.

To prevent the complications of a metagenomics sample, we will work with a different viral read dataset that was sampled directly from an infected cultured cell.

Sequencing a virus is more straightforward than assembling a bacterium for a couple of reasons. First, the virus's genome is much shorter. Second, SARS-CoV-2 is an **RNA virus**, meaning that its genome does not contain DNA, but rather a single (linear) strand of RNA, which is single-stranded; as a result, we do not encounter the complications that we do when assembling double-stranded DNA. Researchers convert RNA to DNA by using an enzyme called **reverse transcriptase**, a molecular tool that was invented by viruses to help them replicate their genome and now has been borrowed by humans to help us sequence this genome.

In this assignment, we will assemble the SARS-CoV-2 genome, and then we will **annotate** the contigs resulting from this assembly, meaning that we will compare our genome against a database of known genomes to label putative genes. Knowing the locations of these genes will allow us to study them in future work.

Getting started with Galaxy

To assemble the genome from the reads, we will be using the [SPAdes assembler](#)

that is built upon the de Bruijn graph approach that we learned about and is the most cited genome assembler of all time. To run SPAdes, we will use the Australian service of Galaxy, an open source project that allows us to run often testy bioinformatics software in the cloud without the hassle of dealing with local installations. Please follow these step-by-step instructions to register on Galaxy and run SPAdes:

First, create an account on Galaxy at <https://usegalaxy.org.au/login>. Your “public name” can be whatever you like, but you should fill out all fields. After creating an account, log in to Galaxy.

Click on the plus ‘+’ on the top right of the page and create a new “history” (i.e., project that will contain data and the results of software runs) named “SARS-CoV-2”. All of the following analysis will be performed under this history.

Grabbing some sequencing data

The National Center for Biotechnology Information (NCBI) hosts an enormous amount of publicly available biological data. You may be interested in their website for SARS-CoV-2 (<https://www.ncbi.nlm.nih.gov/sars-cov-2/>), which contains sequencing data from hundreds of thousands of sequencing “runs”. These runs are stored in an arm of NCBI called the Sequence Read Archive (SRA), where researchers can upload sequencing reads from their experiments.

Our dataset of interest has the SRA identifier SRR11528307, and its homepage can be found here: <https://trace.ncbi.nlm.nih.gov/Traces/sra/?run=SRR11528307>. We can see at this page that the dataset contains approximately 719,800 reads (“spots”), that the technology is “Illumina”, and that the layout is “paired”. These are short paired-end reads that we learned about when discussing how genomes are assembled. **Exercise:** How many nucleotide bases are found in this file? And, what is the GC-content of the reads? (That is, the percentage of reads that are either G or C.)

You should provide an answer to both questions.

The sequence is 186.5 million bases long with 38.2% GC content.

You may recall that when discussing genome assembly, we had a notion of “perfect coverage”, meaning that every k -mer substring from the genome was captured.

In practice, this is a very strong assumption, and instead researchers define the **coverage** of a collection of sequencing reads as the ratio between the total number of nucleotides contained in the reads and the length of the genome.

Exercise: We know that the original SARS-CoV genome from the 2003 outbreak has approximately 30,000 nucleotides. If the SARS-CoV-2 genome has length 30,000 nucleotides, then what is the coverage of this read dataset?

$$186,500,000 / 30,000 = 6,217$$

Clicking on the “Reads” tab will allow us to see ten read-pairs at a time. Each read is in **FASTA format**, meaning that the read has a header beginning with the “>” symbol identifying the read, followed by the read itself.

Exercise: What is the read-pair at spot #15213?

Note that one read is shorter than the other, which could occur for lots of reasons; the most likely one is that the machine at that position started losing signal and the read was cut off.

From the NCBI website, the read pair at spot 15213 is:

>gnl|SRA|SRR11528307.15213.1MN01288:4:000H32WJK:1:11101:22090:17095 *Biological (Biological)*

CAACAAGGCCAAACTGTCACTAAGAAATCTGCTGCTGAGGCTTCTAAGAAGCCTCGGCA
AAAACGTACTGCCACTAAAGCATACAATGTAACACAAGCTTTCGGCAGACGTGGTCCAG
AACAAACCCAAGGAAATTTGGGGACCAGGAA

>gnl|SRA|SRR11528307.15213.2MN01288:4:000H32WJK:1:11101:22090:17095 *Biological (Biological)*

TCAATATGCTTATTCAGCAAAATGACTTGAT

Now that we know where our data is contained, let's import it into Galaxy. To do so, go back to Galaxy, and on the left side of the page, under "Get Data", click "Download and Extract Reads in FASTA/Q Format from NCBI SRA". Make sure that "select input type" shows "SRR accession", and type our accession ID (SRR11528307) into the field below. You may want to click "Yes" for "Email notification", although the job should not take more than five minutes to run. Then click "Execute". You should see the job added to your history on the right; it will turn green when it is finished.

FASTQ format and Phred quality scores

When the job turns green, click on the “eye” symbol next to SRR11528307 under your history on the right side of the page. The page may take a few seconds to load, and Galaxy will show you the first megabyte of your read file.

This file is in **FASTQ format**, an extension of FASTA format in which each read is represented over four lines described as follows.

1. A header beginning with the “@” symbol and labeling the read. Notice that because these are read pairs, the first two reads’ headers are the same and end with “/1” and “/2”.
2. The read as a sequence of nucleotides.
3. A line containing a “+” symbol to indicate the end of nucleotides. This line may have more header information, but doesn’t for this dataset.
4. A collection of ASCII symbols representing **Phred quality scores**, described below. The i -th quality score corresponds to the i -th nucleotide in the read.

A given base returned as the result of a sequencing run is assigned a quality score Q based on an estimate of how likely this base is correct. Because many bases are very reliable, this computation is done on a logarithmic scale. In particular, once we compute the probability p that a base is incorrect, we set $Q = -10\log_{10}(p)$ and assign the base the ASCII symbol corresponding to $Q+33$.

For example, if p is 0.001, then $\log_{10}(p)$ is -3 , and so Q is equal to 30, and we assign this base the ASCII symbol corresponding to $30+33$, which is “?”. As a result, the lower the value of p , the higher the quality score Q . The Phred quality score table is shown in the figure below.

ASCII_BASE=33 Illumina, Ion Torrent, PacBio and Sanger

Q	P_error	ASCII	Q	P_error	ASCII	Q	P_error	ASCII	Q	P_error	ASCII
0	1.00000	33 !	11	0.07943	44 ,	22	0.00631	55 7	33	0.00050	66 B
1	0.79433	34 "	12	0.06310	45 -	23	0.00501	56 8	34	0.00040	67 C
2	0.63096	35 #	13	0.05012	46 .	24	0.00398	57 9	35	0.00032	68 D
3	0.50119	36 \$	14	0.03981	47 /	25	0.00316	58 :	36	0.00025	69 E
4	0.39811	37 %	15	0.03162	48 0	26	0.00251	59 ;	37	0.00020	70 F
5	0.31623	38 &	16	0.02512	49 1	27	0.00200	60 <	38	0.00016	71 G
6	0.25119	39 '	17	0.01995	50 2	28	0.00158	61 =	39	0.00013	72 H
7	0.19953	40 (	18	0.01585	51 3	29	0.00126	62 >	40	0.00010	73 I
8	0.15849	41)	19	0.01259	52 4	30	0.00100	63 ?	41	0.00008	74 J
9	0.12589	42 *	20	0.01000	53 5	31	0.00079	64 @	42	0.00006	75 K
10	0.10000	43 +	21	0.00794	54 6	32	0.00063	65 A			

Figure: The current Phred score table. Each row contains a quality score QQQ , the corresponding probability of error, and the ASCII symbol used to label a nucleotide having this quality score. Source: <https://bit.ly/2NrZvS7>.

Phred scores are helpful for a variety of reasons. For example, researchers will throw out reads having a significant number of bases that do not meet a certain Phred score threshold, especially if the reads have high coverage.

Exercise: How would Phred scores be useful when applying a de Bruijn graph approach to genome assembly? Justify your answer. (Hint: consult our discussion of error detection and removal in de Bruijn graphs.)

We could use Phred scores to filter out bad Phred scores, especially at points in the de Bruijn graph where there is a split/bubble and we need to decide how to consolidate the graph.

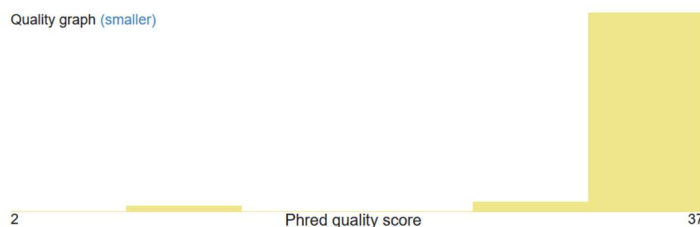
The Illumina sequencing by synthesis approach is described in the following video.

<https://www.youtube.com/watch?v=fCd6B5HRaZ8>

Exercise: How do you think that Illumina determines the quality score of a given base?

Since bases are determined by color in the resulting image, scoring might be based on how “pure” the color at a given spot is. If there is an equal mix of yellow and green signal for example, the two corresponding bases are found in equal amounts. This would correspond to a lower score than a position where the only signal detected is yellow.

Exercise: Head back to the [SRA page](#) for our dataset. In the “Metadata” tab, there is small text that says “Quality graph”. You may need to click “bigger” to see this graph, which is a histogram of Phred quality scores over the entire dataset. Interpret this chart. Are the quality scores good? What makes you think so?



The Phred score histogram shows that the majority of reads have a Phred scores in the furthest right window (ending in 37). This indicates that the bases have low p-value, meaning we are confident that many of these reads are correct.

Exercise: Return to Galaxy and view your SRR11528307 dataset. What are the quality scores of read “8595/1”? Are they good? Are there any nucleotides that you are worried about?

```
@SRR11528307.MN01288:4:000H32WJK:1:11101:10036:8595/1
```

```
TTTATATACTGCTCATACTTTCCAAGTTCTTGGAGATCGATGAGAGATTCATTAAATTCTTGG  
CAACCTCATTGAGGCGGTCAATTTCTTTTGAATG
```

```
+
```

```
F/FFFFFFFFFFFFFFFF/FFFF=FFFFAFFAFFFFFFFFFAAFFFFFFFFFFFFFFFF/FFFFFFFFFFFFFFFF  
FFFAFAF//FFFFFFFFFFFF/F/AFFFF//A
```

Converted to Phred Scores via Python script:

```
>SRR11528307.MN01288:4:000H32WJK:1:11101:10036:8595/1
```

```
37 14 37 37 37 37 37 37 37 37 37 37 37 37 37 37 14 37 37
```

```
37 37 28 37 37 37 37 32 37 37 32 32 37 37 37 37 32 32 32
```

```
37 37 37 37 37 37 37 37 37 37 37 37 14 37 37 37 37 37 37
```

```
37 37 37 37 37 37 37 37 37 37 37 37 32 37 32 37 14 14 37 37
```

```
37 37 37 37 37 37 37 14 37 14 32 37 37 37 37 14 14 32
```

There are many good Phred scores (>30, often 37), but a few bases have a Phred score of 14 (for example the second base), so we are not confident in those nucleotides.

Assembling the SARS-CoV-2 genome

Now that we understand our data a bit better, we are ready to assemble the SARS-CoV-2 genome from our read set using SPAdes. Under "Genomics Analysis" → "Assembly" on the left side of the page, you will see over a dozen different assemblers. Click on "SPAdes", which will take you to its homepage.

We will go through each of the settings offered and briefly explain them as well as what we are going to choose.

- *Single-cell?* Set to "No". This setting is used for bacterial projects when we have a bacterium that we cannot culture, and we only have DNA captured from a single cell.
- *Run only assembly?* Set to "No". We want SPAdes to try and "error correct" reads, meaning that it looks for potentially troubling bases and correcting reads. For example, if a read matches another read exactly except for a single base with a very low quality score, we may want to error correct this read before we build a de Bruijn graph.
- *Careful correction?* Set to "Yes". The BWA tool is based on a Burrows-Wheeler transform approach to read mapping that we learned about when discussing read mapping approaches.
- *Automatically choose k-mer values?* Set to "Yes". SPAdes works by building de Bruijn graphs for several values of k . The default values are $k = 21, 33$, and 55 . We have no *a priori* information that would lead us to change SPAdes' default behavior, so we will let its "machine learning" choose the best values to use.
- *Coverage cutoff?* Set to "Off". If we had a high-coverage collection of reads of varying lengths, we might choose only the longest reads above some threshold (assuming they have sufficient quality) to save us some time in running the assembler. We are assembling a virus, so we don't need a coverage cutoff.
- *Libraries are ionTORRENT reads?* Set to "No" as these are Illumina reads. ionTORRENT reads are produced by a different company (Oxford Nanopore) and are longer and typically less accurate.
- *Library type.* Set to "Paired end/single reads". Remember that we know these are paired-end reads.
- *Orientation.* Set to "fr". Remember that the convention is to always read DNA in the 5' to 3' direction. In forward-reverse read pairs, the first read is taken from one

strand, and its pair is taken from the opposing strand, as the following figure illustrates.

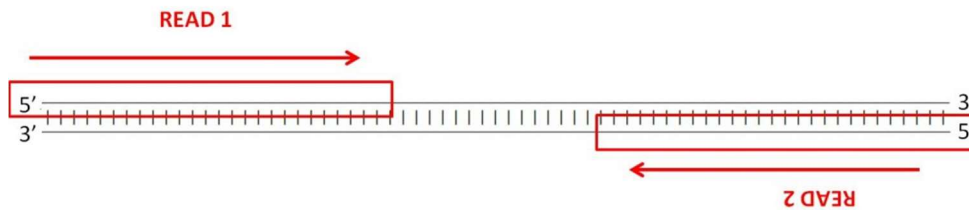


Figure: In “forward-reverse” reads, the first read is taken from one strand, and the second read comes from the opposing strand. Source: <https://bit.ly/3dyv47n>.

Select file format. Set to “interleaved files”. As you know, the read-pairs are found as consecutive reads in our fastq file. This is called “interleaved” as opposed to having the forward reads in one file and the reverse reads in another.

- *Interleaved paired reads.* You should be able to select your SRR11528307 dataset for this field.
- *PacBio reads, etc.* Don’t choose anything in these sections.
- *Output final assembly graph (contigs)?* Set to “Yes” as we would love to know the final assembly graph produced by SPAdes.
- *Output final assembly graph (scaffolds)?* Set to “No”. Scaffolds are formed by trying to join contigs into longer contiguous sequences. We will only view the graph of the contigs.
- *Email notification?* We suggest setting to “Yes” so that you can leave this running and get notified when it’s done. You are now ready to go! Click “Execute”.

SPAdes should take 15-20 minutes to finish.

Analyzing the results of our assembly

Now that SPAdes has completed, we will analyze the results. As the previous exercise indicates, our work appears to have been quite successful.

Exercise: Click on the eye symbol (“view data”) next to the “contigs (fasta)” line on the right side. How many contigs did our assembly produce, and what are their lengths? What do you think “coverage” means in this context?

There are 3 contigs with lengths 20761, 8916, and 147.

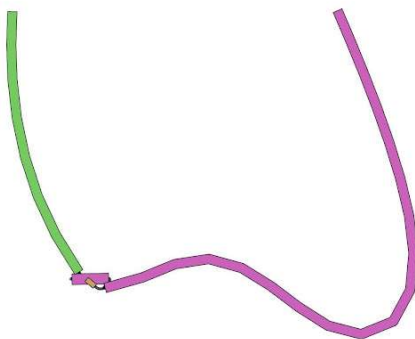
Coverage here is the number of bases in the reads that align with a given contig divided by the total contig length. Again, the higher the coverage the more confident we are that the contig is real.

We also produced an “assembly graph” as the result of our work, which is a version of the de Bruijn graph (after some cleaning steps involving bubble removal) in which maximal nonbranching paths have been compressed to a single edge. If you “view data” for the assembly graph, you will see that this graph has four edges.

Let’s visualize this assembly graph! To do so, we use a program called Bandage. Under “Assembly” on the left side of the page, click on “Bandage Image”. Under “Graphical Fragment Assembly” select the assembly graph that you just produced. Feel free to keep all other parameters the same (possibly opting for an email notification), and click “Execute”. The program should only take a couple of seconds to run, after which clicking “view data” on the run will show the graph.

Exercise: Show the image produced of the assembly graph. Knowing what you know about the two contigs' lengths, where are contig 1 and contig 2 in this Bandage plot?

Contig 1 (length 20761) corresponds to the purple region, contig 2 (length 8916) to the green region, and contig 3 (length 147) to the brown region.



Aligning the two viruses

Because we have a single long contig, and the virus genomes are ultimately short, let's align this contig against the original SARS-CoV genome from the 2003 outbreak to see how the two coronavirus genomes differ.

Exercise: We will align the two genomes as DNA strings rather than translating them to protein strings. Why do you think this is?

Some mutations in DNA still code for the same amino acid, so to better detect differences we need to compare the underlying DNA sequences. Also, when we are analyzing an unknown protein we don't yet know where genes are, or if the sequence codes for proteins at all, so we need to start with the DNA sequence.

To align the SARS-CoV-2 genome against the original SARS-CoV genome, we will use the (affine) global alignment algorithm that we learned about when discussing alignment.

The original SARS-CoV genome is provided at NCBI at <https://bit.ly/2NtCsWX> under the accession ID NC_004718.3. To import this genome, we will use the "NCBI Accession Download" tool under "Get Data" on the left side of the Galaxy page. After clicking on this tool, follow these steps.

- Under "Select source for IDs", select "Direct Entry".
- Under ID List, type NC_004718.3.
- Keep the other parameters the same, add an Email notification if you like, and click "Execute".

The tool should be very quick to run.

Now we are ready to align the two genomes. Under the "EMBOSS" section on the left side of the page, click "needle", which is short for "Needleman-Wunsch global alignment", the algorithm that we learned about for sequence alignment.

For sequence 1, select the contigs that we produced as the result of SPAdes. For sequence 2, select NC_004718.3. Keep the other parameters default (you may like to play around with these parameters later to see how they affect the outcome). After opting for an Email notification, click "Execute". The algorithm should take a few minutes to run.

When the algorithm has finished, view its data. This will show the alignment, in which matches are shown with "|" symbols, mismatches are shown with "." symbols, and indels do not have a symbol between the two rows.

Exercise: How many matched symbols does the alignment report? How many gap symbols?

```
#=====
#
# Aligned_sequences: 2
# 1: NODE_1_length_20761_cov_2834.337024
# 2: NC_004718.3
# Matrix: EDNAFULL
# Gap_penalty: 10.0
# Extend_penalty: 0.5
#
# Length: 30108
# Identity:   17219/30108 (57.2%)
# Similarity: 17219/30108 (57.2%)
# Gaps:       9704/30108 (32.2%)
# Score: 70864.5
#
#
#=====
```

Only NODE_1 was aligned. Out of 30,108 total bases, 17219 are matches and there are 9704 gap symbols.

Exercise: What do you notice about the ends of the alignment? What do you think happened when the genome was assembled?

```
NODE_1_length      0 ----- 0
NC_004718.3        1 ATATTAGGTTTTACCTACCCAGGAAAAGCCAACCAACCTCGATCTCTTG 50
NODE_1_length      0 ----- 0
NC_004718.3        51 TAGATCTGTTCTCTAAACGAACTTTAAATCTGTGTAGCTGTCGCTCGGC 100
NODE_1_length      0 ----- 0

NODE_1_length      20737 -TAGCAGGA-----GAAGTTCCTACTGCT----- 20761
                        .|||.|||          |||          ||.||||
NC_004718.3        29601 GGAGTACGATCGAGGGTACAGTGAA-----TAATGCTAGGGAGAGCTG 29643
NODE_1_length      20761 ----- 20761
NC_004718.3        29644 CCTATATGGAAGAGCCCTAATGTGTAAAATTAATTTTAGTAGTGCTATCC 29693
NODE_1_length      20761 ----- 20761
NC_004718.3        29694 CCATGTGATTTTAAATAGCTTCTTAGGAGAATGACAAAAAAAAAAAAAAAAA 29743
NODE_1_length      20761 ----- 20761
NC_004718.3        29744 AAAAAAAA 29751
```

The ends of the alignment are empty for the contig while the genome continues. This is because the contig is from the middle of the genome. Other contigs might align at the beginning or the end.

Exercise: Scroll through the interior of the alignment. Do you notice any regions that appear to be more variable than other regions? Where?

While many regions like bases 10535-10734 of contig 1 (19452-19651 of the genome) have just a few point mutations, other regions like bases 12994-13079 (21901-21974 of the genome) have large gaps inserted in both the contig and the genome.

```

10535 CTCGATGCTTATAACATGATGATCTCAGCTGGCTTTAGCTTGTGGGTTTA 10584
      .|.|||||.|||||.|||||||.|||.|||||.|||||.|||.||||
19452 TTGGATGCATATAATATGATGATTTCTGCTGGATTAGCCTATGGATTTA 19501

10585 CAAACAATTTGATACTTATAACCTCTGGAACACTTTTACAAGACTTCAGA 10634
      |||||||||||||||||||.|||||.|||.|||||.|||.||||
19502 CAAACAATTTGATACTTATAACCTGTGGAATACATTTACCAGGTTACAGA 19551

10635 GTTTAGAAAATGTGGCTTTTAAATGTTGTAAATAAGGGACACTTTGATGGA 10684
      |||||||||||||||.|||||||.|||||.|||||||||||||
19552 GTTTAGAAAATGTGGCTTATAATGTTGTTAATAAGGACACTTTGATGGA 19601

10685 CAACAGGGTGAAGTACCAGTTTCTATCATTAAATAACACTGTTTACACAAA 10734
      ||...||.|||||.|||.|||||.|||||||.|||.|||||||
19602 CACGCCGGCGAAGCACCTGTTCCATCATTAAATGCTGTTTACACAAA 19651

```

```

12994 TTTTGGGTGTTTAT-----TACCACAAAAACA-ACAAAAGTTGG 13031
      |.|.|.|||||.|          || |||||.||| ||.|
21901 TCTTTGCTGTTTCTAAACCCATGGGTA-CACAGACACATACTA----- 21942

13032 ATGGAAAGTGAGTTCAGAGTTTATTC--TAGTGCGAATAATTGCACTTTT 13079
      |||          ||||| ||.||||...|||||||||||
21943 -----TGA-----TATTCGATAATGCATTTAATTGCACTTTC 21974

```

Annotating the SARS-CoV-2 genome

Finally, we will **annotate** the SARS-CoV-2 genome, meaning that we will identify putative genes and then compare these genes against a database of known genes to find which ones align the best.

The identification of putative genes can be done in a variety of ways. One way to find these regions requires the observation that regions of DNA that are translated into protein start with a **start codon** (ATG) and end with a **stop codon** (TAG, TAA, or TGA). If a genome were random, then after encountering a start codon, we would expect to see a stop codon after about $64/3 \sim 21$ codons. But real genes are typically much, much longer than 21 codons. As a result, in simple organisms like viruses and bacteria, a great way to find genes is to look for long stretches of codons connecting a start codon to a stop codon, with no intermediate stop codons.

Once we have found putative genes along the genome, we can compare them against a database of genes using an algorithm called BLAST that we will learn about soon.

These two steps are taken by our next tool, called Prokka, which is used to annotate the genomes of viruses, bacteria, and archaea. It can be found under “Annotation” on Galaxy. Under “Contigs to annotate”, select our contigs from the SPAdes run. Set “Kingdom” to “Viruses” and leave all other parameters to default. Note that our shorter contig will not be included in the annotation because Prokka only annotates contigs of length at least 200. Select whether you would like an Email on completion, and then click “Execute”.

The entire process of annotating our genome should not take more than a few seconds. Isn't bioinformatics great?

Prokka produces a collection of output files, which you might like to view. The .fna file contains the lone contig that survived. The .gff file contains regions identified by Prokka as genes. The .ffn file contains nucleotide sequences of the annotated genes, and the .faa file contains their translated amino acid sequences.

Exercise: View the .gff file containing regions identified by Prokka as putative genes. How many are there? What is the longest and shortest region?

There are 11 putative genes. The longest is EBFOLEGJ_00002 which spans base 4781 to 12568 (length 7787) and is identified as Replicase polyprotein 1ab. The shortest is EBFOLEGJ_00011 which spans base 7421 to 7561 (length 141) and is identified as a hypothetical protein.

Prokka contains the annotation information in these files, but what a biologist would really like is a visualization of its genes and what they have been predicted to be. So to visualize our annotation contained in the .gff file, we will use a genome browser tool called “JBrowse” that is found in the “Graph/Display” section on the left side of the page.

When running JBrowse, take the following steps.

- *Reference genome to display.* Select “Use a genome from history” and choose the .fna file.
- Click “Insert Track Group”.
- Click “Insert Annotation Track”.
- Select your.gff file from the Prokka output.
- Under “Email notification”, choose “Yes” if you like.
- Then click “Execute”.

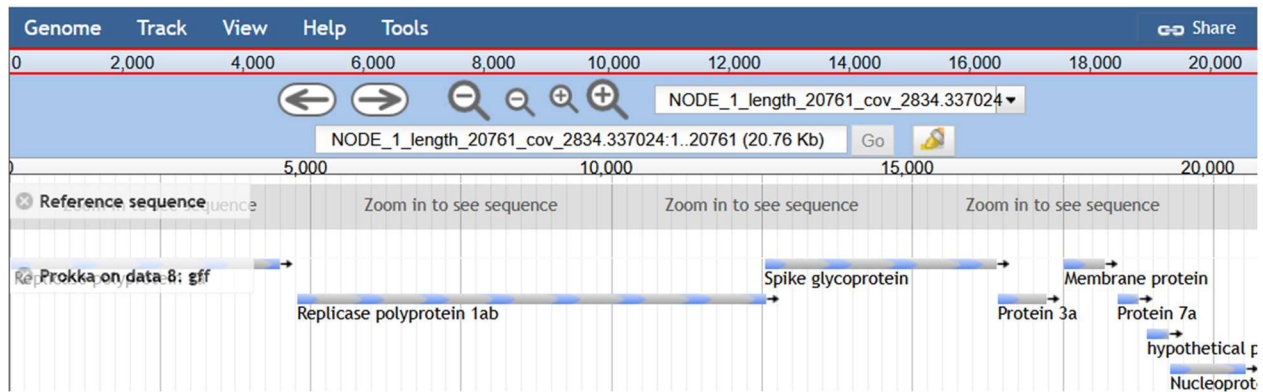
JBrowse should be very quick. When it is finished, click "view file", which is an HTML file that we can view in the browser. (It may take a moment to load.)

It seems like nothing is there, but on the left side of the page, click on “Prokka on data XXX: gff” to show our beautiful annotation of the SARS-CoV-2 genome. Zoom out to see it in all its glory. All the arrows point in the same direction to indicate that the genes are all found on the same strand of the genome. This makes sense because SARS-CoV-2 is an RNA virus, meaning that its genome has only one strand. (It would have been a very bad sign if some genes pointed in the opposite direction.)

You can click on the genes as you like to obtain information about their length and what in the database they were found to be similar to (if anything). For example, if you click on the first protein (Replicase polyprotein 1a), you will find that it was found to be similar to the protein with Uniprot ID (<https://www.uniprot.org/uniprot/POC6U8>), which is the same gene in SARS-CoV. Note that the annotation score of this protein is high because there is so much experimental evidence backing up this protein. This means that if we find a putative gene that is a hit against it, we can have a high degree of certainty that our gene serves a similar function.

Exercise: Why do you think that two of the genes are labeled “hypothetical proteins”?

These are likely protein predictions that exist in Uniprot but do not yet have an experimental validation.



It may amaze you that such a tiny thing can wreak so much havoc. But now that we know this annotation, we can start investigating the virus's individual genes. Which one should we focus on? How has the gene mutated as the virus spread around the world? And how do these mutations affect the function of the protein? We save these topics for future study in this capstone.